

L12a: Model-Free Reinforcement Learning

Q-learning

CHEME 5800 Cornell University

Copyright Jeffrey Varner 2026. All Rights Reserved

Objectives for Today

How can an agent learn to act well when it has no model of its environment?

Today's concepts

- Describe the reinforcement learning problem: an agent takes an action, then sees a reward and a new state, without knowing the rules.
- Update action values from experience: after each step, Q-learning corrects one action value toward what the agent observed.
- Explain when Q-learning converges: Q-learning needs every action tried again and again, and a learning rate that shrinks at the right pace.

Lecture: Model-Free Reinforcement Learning and Q-Learning

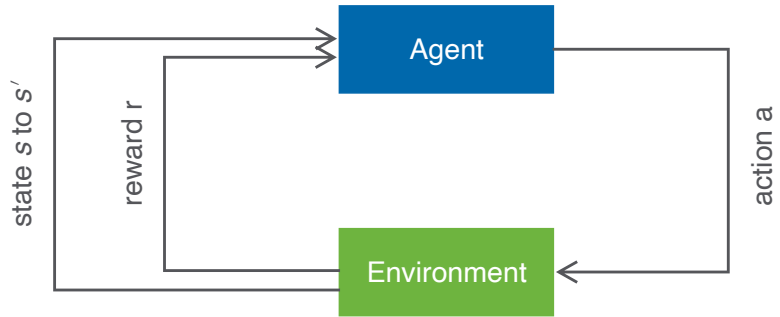
Lecture and Example

Lecture: Model-Free Reinforcement Learning and Q-Learning
In this lecture, we write the Q-learning update rule, assemble it into the Q-learning algorithm, and state when Q-learning converges.

Example: Q-Learning in the Lava Grid World
In this example, a roomba that has finished cleaning the kitchen learns to return to its charging station without falling into a lava pit. We train the roomba with Q-learning, then compare its policy with the optimal policy from value iteration.

The Reinforcement Learning Problem

How does an agent learn the best action in each state when it does not know the rules of its world?



The agent observes its current state s and selects an action a. The environment answers with a reward r and the next state s'.

Learning from Interaction

We have seen two approaches to learning from interaction:

- Multiplicative weights adjusts the probability of each action from its past performance. It does nearly as well as the best fixed action in hindsight.
- Bandit algorithms work without states. In each round, they estimate the reward of each action and adapt how they choose.
- Q-learning adds the state. It estimates the long-term value of each state-action pair, which lets the agent learn when its actions change what happens next.

All three balance exploring new actions to gather information against exploiting known actions to collect reward.

From Value Iteration to Action Values

A Markov decision process (MDP) is the tuple $(\mathcal{S}, \mathcal{A}, R(s,a), T(s' \mid s,a), \gamma)$ of states, actions, rewards, transition probabilities, and a discount factor. Value iteration computes the optimal value function $U^*(s)$ by repeating the Bellman backup in sweeps k:

$$U_{k+1}(s) = \max_{a \in \mathcal{A}} \Big(\underbrace{R(s,a)}_{\text{now}} + \gamma \overbrace{\sum_{s' \in \mathcal{S}} T(s' \mid s,a)\, U_k(s')}^{\text{future}} \Big)$$

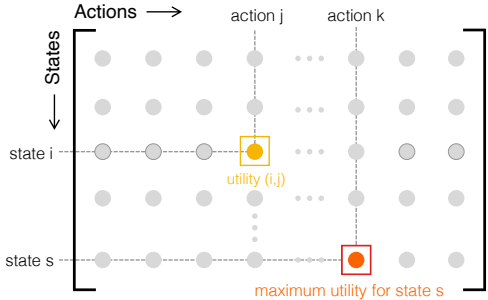
We write the action-value function, also called the Q-function, in terms of U^*:

$$Q^*(s,a) = R(s,a) + \gamma \sum_{s' \in \mathcal{S}} T(s' \mid s,a) \underbrace{U^*(s')}_{= \max_{a' \in \mathcal{A}} Q^*(s',a')}$$

$Q^*(s,a)$ is the value of taking action a in state s: the reward now plus the discounted expected value of the next state.

The Q-Table

We store the action values in a table, the Q-table, with one row per state and one column per action.



The policy reads each row and picks the column with the largest entry.

What Value Iteration Needs

Value iteration uses the reward function $R(s, a)$ and the transition model $T(s' \mid s, a)$:

- With R and T, value iteration requires $\mathcal{O}(|\mathcal{S}|^2 \, |\mathcal{A}|)$ operations per sweep.
- Without R and T, the usual case for a real robot, a reactor, or a patient, what can we do?

Let's think of the Q-table as the agent's memory. The agent can then learn to act optimally by updating its memory after each step from the outcomes of its actions, without T or R.

The Q-Learning Update

Q-learning (Watkins, 1989) estimates $Q(s, a)$ from repeated experiments. In each experiment, the agent in state s takes action a, receives a reward r, and moves to a state s'.

After each experiment, the agent updates one entry of the Q-table:

$$Q(s,a) \leftarrow \underbrace{Q(s,a)}_{\text{old value}} + \alpha \cdot \underbrace{\left(\overbrace{r + \gamma \cdot \max_{a' \in \mathcal{A}} Q(s',a')}^{\text{target}} - Q(s,a) \right)}_{\text{net new information}}$$

where $0 < \alpha \leq 1$ is the learning rate and $0 \leq \gamma < 1$ is the discount factor. Every other entry keeps its value.

The Update as a Weighted Average

The update is also a weighted average of the old value and the target:

$$Q(s,a) \leftarrow (1-\alpha) \cdot \underbrace{Q(s,a)}_{\text{old value}} + \alpha \cdot \underbrace{\left(r + \gamma \cdot \max_{a' \in \mathcal{A}} Q(s',a') \right)}_{\text{target}}$$

We estimate the policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$ by selecting the action that maximizes $Q(s,a)$ in each state s:

$$\pi(s) = \arg\max_{a \in \mathcal{A}} Q(s,a)$$

Episodes and Epsilon-Greedy Actions

Q-learning runs in episodes. Each episode runs until the agent enters an absorbing state or reaches a step limit.

Each step chooses an action with the epsilon-greedy rule:

- With probability ϵ, the agent explores with a random action.
- Otherwise, the agent exploits the action with the largest current value.

Related to bandits? Each row of the Q-table acts like its own bandit, and epsilon-greedy is a bandit rule. Other bandit rules, such as UCB1, can replace it, as long as every action in every state keeps being tried.

Q-Learning Algorithm

Initialize: Given $\mathcal{S}$, $\mathcal{A}$, the absorbing states, $0 \leq \gamma < 1$, $0 < \alpha \leq 1$, $0 \leq \epsilon \leq 1$, a number of episodes $N \geq 1$, and a step limit $d \geq 1$, set $Q(s, a) \leftarrow 0$ for all s and a.

For $i = 1$ to N **do**:

1. Choose a starting state s that is not absorbing, and set $t \leftarrow 0$.
2. While s is not absorbing and $t < d$ **do**:
   - Choose the action a: roll a random number $p \in [0, 1]$.
     - If $p \leq \epsilon$ **then** choose $a \in \mathcal{A}$ uniformly at random (exploration).
     - Otherwise, choose $a \leftarrow \arg\max_{a' \in \mathcal{A}} Q(s, a')$ (exploitation).
   - Take action a, and observe r and s' from the world.
   - Update $Q(s, a) \leftarrow Q(s, a) + \alpha\,(r + \gamma \max_{a' \in \mathcal{A}} Q(s', a') - Q(s, a))$.
   - Update $s \leftarrow s'$ and $t \leftarrow t + 1$.

Extract Policy: for each $s \in \mathcal{S}$, compute $\pi(s) \leftarrow \arg\max_{a \in \mathcal{A}} Q(s, a)$.

Absorbing States

An episode ends as soon as the agent enters an absorbing state:

- The agent never updates the rows of Q that belong to absorbing states. Those rows stay at zero.
- The target of a move into an absorbing state is just its reward r, with no future to add.

When Does Q-Learning Converge?

Q-learning converges to the optimal action values, $Q(s,a) \to Q^*(s,a)$, with probability one (Watkins and Dayan, 1992). The environment must have the Markov property, and three conditions must hold:

- Finite, fixed MDP and discounting: the states and actions are finite, the rewards are bounded, the rewards and transition probabilities do not change, and $0 \leq \gamma < 1$.
- Learning rate decay: the learning rate $\alpha_n(s,a)$ of the n-th update of each pair satisfies $\sum_{n=1}^{\infty} \alpha_n(s,a) = \infty$ and $\sum_{n=1}^{\infty} \alpha_n^2(s,a) < \infty$.
- Infinite exploration: every state-action pair of a state that is not absorbing is visited, and updated, infinitely often.

Meeting the Conditions

- The first sum lets the updates carry $Q(s, a)$ as far from its starting value as it needs to go. The second sum makes the noise of the individual targets average out.
- The choice $\alpha_n(s, a) = 1/n$ satisfies both sums. With this choice, $Q(s, a)$ is the running average of the targets the pair has seen.
- Epsilon-greedy selection with a fixed $\epsilon > 0$, together with episodes that start at random in any state that is not absorbing, meets the exploration condition.

The greedy policy is already correct once the best action in each state has the largest estimate.

Caveats and Practical Notes

Of the convergence conditions, infinite exploration is the hardest to meet in practice:

- Exploration schedule: if ϵ is too small, or shrinks too quickly, the agent can settle on a poor action. If ϵ stays large, the agent wastes many steps on random actions.
- Sparse and delayed rewards: when few pairs earn a nonzero reward, a new agent may take many steps before its first reward. Later visits can carry that reward back one step at a time. Reward shaping adds small rewards that guide the agent toward the goal.
- Rate of convergence: every state-action pair needs many visits, and the number of experiments grows with the size of the table. When R and T are known, value iteration is usually faster.

Example: Q-Learning in the Lava Grid World

In this example, a roomba that has finished cleaning the kitchen must return to its charging station without falling into a lava pit:

1. Build the lava grid world, and a function that returns the next square and the reward of one move.
2. Train the roomba with Q-learning.
3. Compare the learned policy with the optimal policy from value iteration, and follow both from a starting square.

After 2000 episodes, both policies reach the station from $(20, 20)$ in 33 moves, by different routes.

Example: Q-learning in the lava grid world

Lab: Adaptive Drug Dosing with Q-Learning

In the lab, we treat a course of chemotherapy as a Markov decision process in which we choose a drug dose each treatment cycle:

- The state is the size of the tumor and the level of toxicity, the damage the drug does to healthy tissue. Each action is a drug dose.
- Value iteration computes the optimal dosing policy, which serves as the benchmark.
- You will write the Q-learning update, train an agent on simulated treatment cycles, and compare its doses with the optimal ones.

Lab: Adaptive drug dosing with Q-learning

Summary

We developed model-free reinforcement learning, in which an agent learns how to act from the rewards it receives.

Key takeaways

- **Model-free learning.** The agent learns from the states, actions, and rewards it observes, without a model.
- **Q-learning.** Each step moves one action value toward a target built from the observed reward and next state.
- **Convergence.** Every pair needs visits again and again, and a learning rate that shrinks at the right pace.

Reinforcement learning lets an agent learn good behavior through trial and error, even when no one can write down the rules of its environment.